Simplest Notes — Design

Status: **Proposed** (awaiting approval) · Date: 2026-10-16 · Owner: Gautham

1. Requirements (from the sketch)

Functional

ID	Requirement
R1	Single tap opens the app straight into editing.
R2	Plain text editor only — no markdown, markup, or formatting of any kind.
R3	No save button, no file names, no locations. Text saves as you type.
R4	"Delete" is just Select All (Ctrl/Cmd+A) + Delete. No delete feature exists.
R5	The text is stored in the user's Google Drive.
R6	Runs entirely on-device. No backend server of our own.
R7	First launch: prompt to connect Google Drive and explain why it is useful (Connect / Not now).
R8	After first launch (and after onboarding): open directly to the text interface; on iOS/iPadOS the keyboard is already up.
R9	Top-right corner: small Google Drive icon. **Dull** = not connected, **active** = connected.
R10	Tap icon when not connected → try connecting again.
R11	Tap icon when connected → show connected account email, a **Disconnect** button and a **Switch account** button.
R12	If the app is deleted: text survives in Drive if connected; if never connected, text is gone (expected).

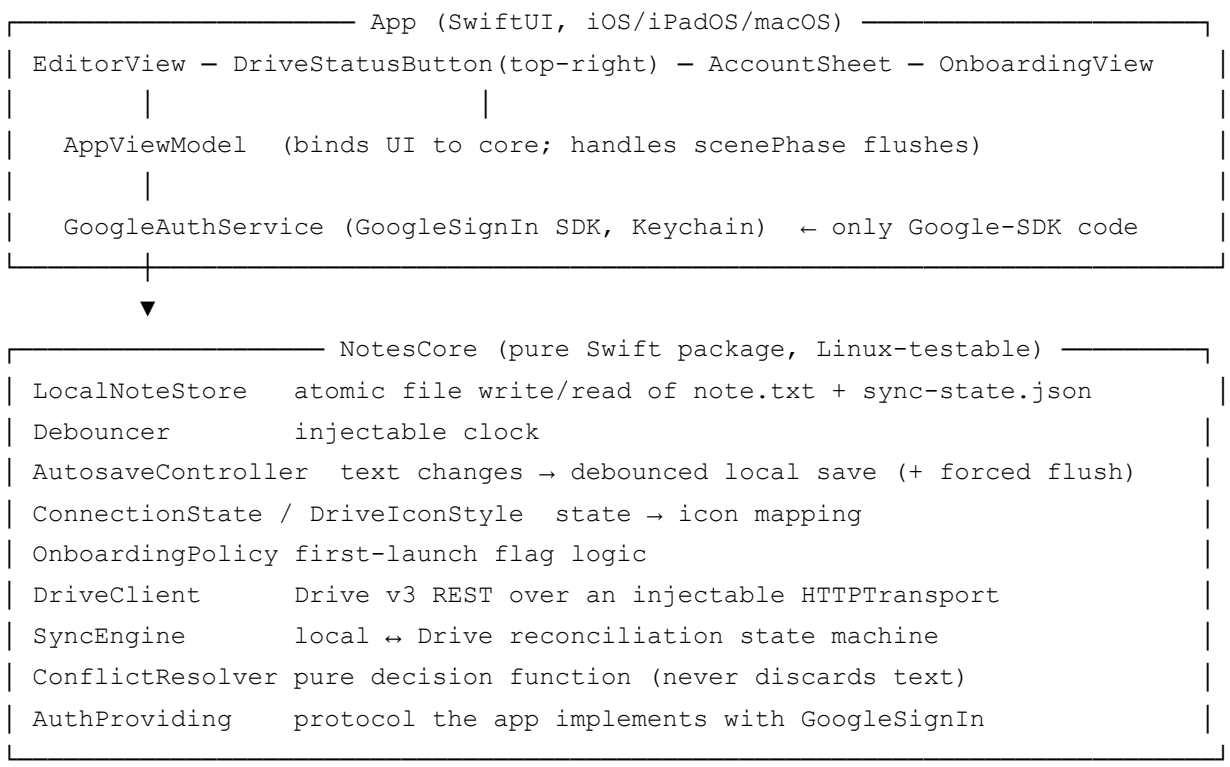
Non-functional

- **N1 Never lose text.** Every keystroke is durably on disk within ~300 ms; Drive is eventually consistent with the device.
- **N2 Offline-first.** Editing never waits on the network.
- **N3 Privacy.** Least-privilege Drive scope; the app can see only files it created.
- **N4 Testability.** Every function has edge-case tests written before its implementation.

Assumptions (please confirm)

- **A1 One note.** "No file names" + "delete = select all" implies a single continuous text. Multiple notes are out of scope.
- **A2 Platforms:** iPhone, iPad and Mac (one SwiftUI codebase). See ADR-0001.
- **A3** Multi-device use (e.g. iPhone + iPad on the same Google account) should work; conflicts must never silently discard text.

2. Architecture



All logic with branches lives in **NotesCore** behind protocols, so it is unit-tested without a device, network, or Google account. The app layer is thin glue, covered by XCUITests with a fake auth/Drive.

3. Key flows

Launch → read `note.txt` → show `EditorView` with focus (keyboard up on iOS/iPadOS) → if first launch, present `OnboardingView` over it → if connected, `SyncEngine.pull()` in background.

Typing → `AutosaveController` debounces (300 ms) → `LocalNoteStore.save` (atomic) → marks `dirty` → `SyncEngine` debounces push (2 s) → upload if remote unchanged, else `ConflictResolver`.

App backgrounded / quit → flush local save immediately → attempt push inside a background task.

Foreground → pull: if remote changed and local not dirty → replace editor text (cursor preserved where possible); if both changed → conflict policy (ADR-0005).

Connect (onboarding, or tapping dull icon) → Google sign-in sheet → locate/create `Simplest Notes/Notes.txt` → initial reconciliation (ADR-0005 §first-connect) → icon turns active.

Disconnect → stop sync, revoke token, sign out; local text stays on device; icon dull.

Switch account → disconnect → sign in with a different account → first-connect reconciliation against that account's file.

4. Data on device

- `Application Support/note.txt` — the text (UTF-8).
- `Application Support/sync-state.json` — `{ driveFileId, lastSyncedRemoteVersion, lastSyncedHash, dirty, accountEmail }`.
- `UserDefaults` — `hasCompletedOnboarding`.
- Keychain (managed by GoogleSignIn SDK) — OAuth tokens.

5. Data in Drive

- Folder `Simplest Notes` containing `Notes.txt` (`text/plain`). Visible to the user in Drive, survives app deletion.
- Conflict copies (rare): `Notes (conflict 2026-10-16 14-03 iPad).txt` in the same folder.

6. Out of scope

Multiple notes, search, formatting, sharing, Android/Windows, any server.

7. ADR index

- ADR-0001 Platform: native SwiftUI multiplatform
- ADR-0002 Single note, local-first storage
- ADR-0003 Serverless Google auth (GoogleSignIn, PKCE, Keychain)
- ADR-0004 Drive location & `drive.file` scope
- ADR-0005 Sync & conflict policy
- ADR-0006 Disconnect / switch-account semantics
- ADR-0007 Testing strategy (TDD, pure core, CI on macOS)

ADR-0001: Native SwiftUI multiplatform app (iOS, iPadOS, macOS)

Status: Proposed · 2026-10-16

Context

The sketch requires the keyboard to already be up on iOS/iPadOS at launch (R8), a single tap to start editing (R1), Google Drive storage (R5), and no server (R6).

Options

Option	Keyboard auto-shown on iOS	Stays signed in w/o server	Testable here
A. Native SwiftUI	Yes (`@FocusState` on appear)	Yes — GoogleSignIn SDK keeps a refresh token in Keychain	Core logic: yes (`swift test`); UI: on macOS CI
B. PWA (static web app)	**No** — iOS Safari refuses programmatic focus without a tap	**No** — browser OAuth gives 1-hour tokens, no refresh token without a server; silent renewal needs popups that iOS standalone PWAs block	Yes

C. Capacitor (web in native shell)	Only via WKWebView private-API workaround	Needs native Google plugin anyway	Partly

Decision

Option A. It is the only option that satisfies R8 and R5+R6 together without hacks. One SwiftUI codebase targets iPhone, iPad and Mac.

Consequences

- Requires Xcode (Mac) to build/install; free Apple ID installs for 7 days, a paid developer account ($99/yr) for permanent install / TestFlight.
- No Android/Windows/web. Revisit only if needed.
- Branching logic goes in a pure Swift package so it is tested on Linux and macOS (ADR-0007).

ADR-0002: One plain-text note, local-first

Status: Proposed · 2026-10-16

Context

R2–R4 forbid formatting, file names, save buttons and a delete feature. R6/N2 require offline editing.

Decision

- The app holds exactly **one** UTF-8 plain-text note.
- The on-device file `note.txt` is written on every edit (debounced 300 ms, atomic write via temp file + rename) and is the source of truth while editing. Drive is a synced replica.
- Editor is SwiftUI `TextEditor` bound to a `String` — no attributed text, no rich paste (pasted formatting is stripped by construction).
- Forced flush on `scenePhase` → `.inactive` / `.background` and on macOS window close / app quit.

Consequences

- Never-connected users lose text if they delete the app (accepted, R12).
- Crash between keystroke and debounce loses ≤300 ms of typing; acceptable.
- Very large text (multi-MB) still works; editor performance is a Phase 7 hardening check.

ADR-0003: Serverless Google auth with GoogleSignIn SDK

Status: Proposed · 2026-10-16

Context

R6 forbids a backend. Drive access needs OAuth 2.0.

Decision

- Use Google's official **GoogleSignIn-iOS** SDK (supports iOS and macOS). It implements the OAuth 2.0 installed-app flow with **PKCE**; no client secret, no server.
- Tokens (incl. refresh token) are stored in Keychain by the SDK; `restorePreviousSignIn()` on launch.
- Scopes: `https://www.googleapis.com/auth/drive.file` + `email` (to display the account, R11).
- Auth sits behind an `AuthProviding` protocol in NotesCore; the app implements it with the SDK; tests use a fake.

Consequences

- **User action needed:** create a Google Cloud project, enable the Drive API, create an *iOS* OAuth client ID, configure the consent screen (External, Testing, add yourself as test user). `drive.file` is a non-sensitive scope, so no Google verification is required for personal use.
- In "Testing" publishing status, refresh tokens expire after 7 days → re-sign-in weekly. Moving to "In production" (no verification needed for non-sensitive scopes) removes that.

ADR-0004: Drive location and least-privilege scope

Status: Proposed · 2026-10-16

Context

R5 says the file lives in the user's Drive; R12 says it must survive app deletion; N3 asks for least privilege.

Options

- `appDataFolder` : hidden from the user — fails "the file resides on GDrive" in a useful sense.
- Full `drive` scope: restricted scope, needs Google security assessment — overkill.
- **`drive.file`** + visible file: app sees only files it created.

Decision

Visible folder **`Simplest Notes`** in My Drive containing **`Notes.txt`** (`text/plain`), created by the app under `drive.file` .

- `driveFileId` is cached in `sync-state.json` .
- On (re)connect or if the cached id 404s/is trashed: search `name='Notes.txt' and '<folderId>' in parents and trashed=false` (only app-created files are visible). If several exist, pick the most recently modified and log the others.
- If none exist: create folder (if missing) then file.

Consequences

- After reinstall, the app finds its old file again because it is the same OAuth client.
- If the user renames/moves the file in Drive, the cached id still works; if they delete it, the app recreates it from local text.
- If the user hand-creates a `Notes.txt` in Drive, the app cannot see it (by design).

ADR-0005: Sync and conflict policy — never discard text

Status: Proposed · 2026-10-16

Context

Same account may be used on several devices, and edits happen offline. N1: never lose text.

Decision

State kept per device: `lastSyncedRemoteVersion` (Drive file `version`), `lastSyncedHash` (SHA-256 of last synced text), `dirty`.

Push (2 s debounce after local save, and on background): fetch remote metadata →

- remote `version` == `lastSyncedRemoteVersion` → upload; record new version/hash; `dirty=false`.
- remote changed → go to *Reconcile*.

Pull (on launch, foreground, after connect): fetch metadata → if version unchanged, nothing. Else *Reconcile*.

Reconcile (pure `ConflictResolver.decide(local, remote, base)`):

local changed since base	**remote changed since base**	**Action**
no	no	nothing
no	yes	take remote into editor
yes	no	upload local
yes	yes, texts identical	mark synced
yes	yes, differ	keep **remote** as `Notes.txt` in editor; upload local as a **conflict copy** file; show a one-line, dismissible banner "Edits from another device conflicted — your version saved as …"

First connect / switch account (no base):

local empty	remote missing/empty	Action
any	missing	create remote from local
yes	exists	take remote
no	exists, identical	mark synced
no	exists, differs	take remote; save local as conflict copy

Network errors: keep `dirty=true`, retry with exponential backoff (2 s → 5 min cap) and on next foreground. 401 → one silent token refresh, then show icon dull-with-warning and keep editing locally.

Consequences

- No 3-way text merge (kept simple); worst case the user sees a second file in Drive.
- Small race between metadata check and upload (Drive v3 has no reliable If-Match on upload); a later pull detects it via version mismatch and applies the table.

ADR-0006: Disconnect and switch-account semantics

Status: Proposed · 2026-10-16

Decision

- **Disconnect:** flush pending push (best effort, 5 s cap), revoke token, sign out, clear `driveFileId`/sync base, keep `note.txt` on device. Icon becomes dull. Drive file is left untouched.
- **Switch account:** Disconnect (as above) then start sign-in. If the user cancels the sign-in sheet, remain disconnected (not reconnected to the old account). On success, run *first-connect* reconciliation (ADR-0005) against the new account.
- **Reconnect after disconnect:** same as first connect.
- Account sheet shows the email from the signed-in user; if unavailable, shows "Google account".
- Sign-in cancelled or failed during onboarding/icon tap → stay disconnected, no error

dialog for cancel; short inline message for real failures.

ADR-0007: Testing strategy

Status: Proposed · 2026-10-16

Decision

- **Test-first per phase:** each phase lists edge cases → tests written and failing → implementation → green.
- **NotesCore** (pure Swift package, no UIKit/AppKit/Google SDK): `swift test` with XCTest. Clock, file system root, HTTP transport and auth are injected, so tests are deterministic and offline.
- **App layer:** XCUITests on iOS Simulator (iPhone + iPad) and macOS, with launch arguments that swap in `FakeAuth` and `FakeDriveTransport`.
- **CI:** GitHub Actions — `swift test` on `ubuntu-latest` and `macos-latest`; `xcodebuild test` for iOS/iPadOS simulators and macOS. Project file generated with XcodeGen (`project.yml`) to avoid hand-editing `.pbxproj`.
- Real Google/Drive calls are exercised only by a manual smoke checklist (no live credentials in CI).
- Coverage target: ≥90 % lines in NotesCore.

Simplest Notes — Phased Development Plan

Status: **Awaiting approval** · 2026-10-16

Every phase follows the same order: **(a) test plan with edge cases → (b) write tests, confirm they fail → (c) implement → (d) green + CI + commit.** Test IDs are referenced in test names (e.g. `test_T1_03_saveIsAtomicWhenWriteInterrupted`).

Phase 0 — Repository & scaffolding

Tests first

- T0.01 `swift test` runs a trivial test on Linux and macOS (CI wiring proof).

- T0.02 iOS, iPadOS and macOS app targets build and launch to a blank screen in UI test.

Implementation: GitHub repo, `NotesCore` Swift package, XcodeGen `project.yml` (iOS 17 / macOS 14), GitHub Actions workflow, README, LICENSE, docs.

Exit: CI green on all jobs.

Phase 1 — Local persistence & autosave (R3, N1, N2)

1A `LocalNoteStore`

Edge-case tests

- T1.01 Load when no file exists → `""` (first launch).
- T1.02 Save then load round-trips exactly (incl. trailing newline, `\r\n`, tabs).
- T1.03 Unicode: emoji, combining marks, RTL, CJK, zero-width joiners round-trip byte-exact.
- T1.04 Empty string saves and loads as `""` (select-all + delete, R4).
- T1.05 Whitespace-only text is preserved, not trimmed.
- T1.06 Large text (5 MB) round-trips.
- T1.07 Atomicity: simulated failure mid-write leaves the previous content intact.
- T1.08 Directory missing → created on first save.
- T1.09 Corrupt/non-UTF-8 file on disk → load returns lossy-decoded text and backs the raw file up as `note.corrupt-<ts>.bin` (never silently drops data).
- T1.10 Read-only / disk-full error surfaces as a typed error, original file untouched.
- T1.11 Sync-state JSON: missing → defaults; corrupt → defaults + `dirty=true` (forces a safe reconcile); round-trips all fields.

1B `Debouncer` (injected clock)

- T1.12 Single call fires once after the delay.
- T1.13 Burst of calls fires once, with the **last** value.
- T1.14 Calls spaced beyond the delay each fire.
- T1.15 `flush()` fires the pending value immediately and cancels the timer.
- T1.16 `flush()` with nothing pending does nothing.
- T1.17 `cancel()` drops pending value.
- T1.18 Delay of zero fires synchronously-next-tick.

- T1.19 Deallocation with pending work does not crash or fire afterwards.

1C `AutosaveController`

- T1.20 Typing triggers exactly one save per debounce window.
- T1.21 Text identical to last saved → no write.
- T1.22 Background/quit event flushes pending text immediately.
- T1.23 Save failure retries on next change and reports `saveError` state; recovers when write succeeds.
- T1.24 Every successful save marks sync state `dirty=true` and emits a "local changed" event.

Implementation: the three types above in NotesCore.

Phase 2 — Editor UI & first launch (R1, R2, R7, R8)

2A `OnboardingPolicy`

- T2.01 Fresh install → show onboarding.
- T2.02 After "Connect" success → never show again.
- T2.03 After "Not now" → never show again (icon remains the path to connect).
- T2.04 After Connect **cancelled/failed** → onboarding marked done, app is in not-connected state.
- T2.05 Reinstall (UserDefaults wiped) but Keychain still has a Google session → skip onboarding, restore session silently.

2B UI tests (XCUITest, fake auth)

- T2.06 Launch (not first) → editor visible and focused; on iOS/iPadOS the software keyboard is shown without any tap.
- T2.07 First launch → onboarding text explains the benefit (backup, survives app deletion, other devices) with Connect / Not now.
- T2.08 After onboarding dismissal → editor focused, keyboard up.
- T2.09 Editor renders no formatting toolbar, no title, no file name, no save button.
- T2.10 Pasting rich text (bold/links) inserts plain characters only.
- T2.11 Cmd/Ctrl+A then Delete empties the note and the empty note persists after relaunch.
- T2.12 Type → kill app → relaunch → text present.

- T2.13 iPad external keyboard and Mac: typing works immediately, shortcuts work.
- T2.14 Rotation / Split View / Stage Manager resize keeps text and cursor.
- T2.15 Dynamic Type & dark mode render legibly; VoiceOver reads the editor and the Drive button label.
- T2.16 Typing at the end of a long note keeps the cursor visible above the keyboard.

Implementation: `EditorView`, `OnboardingView`, `AppViewModel`, scenePhase flush wiring.

Phase 3 — Connection state, Drive icon, account sheet (R9–R11) with fake auth

3A `ConnectionState` → icon mapping

- T3.01 `.notConnected` → dull icon, label "Connect Google Drive".
- T3.02 `.connecting` → dull icon + spinner, taps ignored (no double sign-in).
- T3.03 `.connected(email)` → active icon, label includes email.
- T3.04 `.connected` + sync error/offline → active icon with small warning badge.
- T3.05 `.authExpired` → dull icon with warning; tap re-auths.

3B Interaction state machine

- T3.06 Tap when not connected → starts sign-in.
- T3.07 Sign-in cancelled → back to `.notConnected`, no error shown.
- T3.08 Sign-in fails (network) → `.notConnected` + inline message.
- T3.09 Tap when connected → account sheet shows email, Disconnect, Switch account.
- T3.10 Email missing from profile → sheet shows "Google account".
- T3.11 Very long email truncates in the middle, full text in accessibility label.
- T3.12 Disconnect → `.notConnected`, sheet closes, local text unchanged.
- T3.13 Switch account → disconnect then sign-in; cancel leaves `.notConnected` (not the old account).
- T3.14 Rapid repeated taps on Disconnect/Switch don't run the action twice.

Implementation: `ConnectionState`, `ConnectionController`, `DriveStatusButton`, `AccountSheet`, `AuthProviding` protocol, `FakeAuth`.

Phase 4 — Real Google sign-in (ADR-0003)

Prerequisite (you): Google Cloud project, Drive API enabled, iOS OAuth client ID, consent screen with you as test user. I'll write the exact steps in `docs/SETUP.md`.

Tests first

- T4.01 `GoogleAuthService` maps SDK results → `AuthProviding` results (success, cancel, network error, scope declined) — via a thin SDK adapter protocol so mapping is unit-tested.
- T4.02 Scope declined (user unticks Drive) → treated as failure, message asks to allow Drive.
- T4.03 `restorePreviousSignIn` success at launch → `.connected` without UI.
- T4.04 Restore fails (revoked externally) → `.notConnected` silently.
- T4.05 Access token refresh failure → `.authExpired`.
- T4.06 Disconnect calls SDK `disconnect` (revoke) and `signOut`; revoke failure still signs out locally.
- Manual smoke: real sign-in on iPhone, iPad, Mac.

Implementation: GoogleSignIn SPM dependency, URL scheme, `GoogleAuthService`.

Phase 5 — Drive client (ADR-0004), mocked HTTP

Edge-case tests (`FakeHTTPTransport`)

- T5.01 Find folder: exists → id; missing → nil; multiple → oldest.
- T5.02 Create folder then file with correct metadata/MIME (`text/plain`, parent).
- T5.03 Find file: none / one / multiple (most recently modified chosen) / trashed ignored.
- T5.04 Download returns exact bytes as UTF-8 (incl. empty file, BOM stripped, non-UTF-8 → lossy + flag).
- T5.05 Upload (multipart / media) sends exact body and returns new `version`.
- T5.06 Metadata fetch returns `version`, `modifiedTime`, `trashed`.
- T5.07 Cached file id returns 404 or `trashed=true` → caller gets `.fileGone`.
- T5.08 401 → `.unauthorized`; 403 rate-limit/ `userRateLimitExceeded` → `.rateLimited(retryAfter)`; 403 storage quota → `.quotaExceeded`; 5xx → `.transient`; timeouts/offline → `.offline`.
- T5.09 Query strings are escaped (names with quotes/backslashes).

- T5.10 Auth header uses a fresh token each request.

Implementation: `DriveClient`, `HTTPTransport`, `URLSessionTransport`.

Phase 6 — Sync engine & conflicts (ADR-0005, ADR-0006)

6A `ConflictResolver` (pure; table-driven tests covering every row of both ADR-0005 tables)

- T6.01–T6.05 Steady-state table rows (5 cases).
- T6.06–T6.09 First-connect table rows (4 cases).
- T6.10 Texts differing only by line endings are treated as different (no normalisation surprises) — documented choice.
- T6.11 Conflict copy name includes timestamp + device name, is unique when called twice in the same minute, and sanitises characters Drive dislikes.

6B `SyncEngine` (fake Drive + fake clock)

- T6.12 Local edit → push after 2 s debounce; burst of edits → one upload.
- T6.13 Background event → immediate push.
- T6.14 Offline → stays `dirty`; backoff 2 s, 4 s, … capped at 5 min; reconnect triggers push.
- T6.15 Pull on launch/foreground with unchanged version → no download.
- T6.16 Remote changed, local clean → editor text replaced; cursor position clamped to new length.
- T6.17 Remote changed while user is mid-typing (dirty) → conflict path, no keystrokes lost.
- T6.18 File deleted/trashed in Drive → recreated from local text, new id cached.
- T6.19 401 → one refresh + retry; second 401 → `.authExpired`, edits continue locally.
- T6.20 Quota exceeded → warning badge, local saving unaffected.
- T6.21 Disconnect during in-flight upload → upload cancelled or completed, never writes to the new account after switch.
- T6.22 Switch account → first-connect table against new account; old account's file untouched.
- T6.23 Push and pull never run concurrently (serialised).
- T6.24 App killed between local save and push → next launch sees `dirty=true` and pushes.

- T6.25 Upload succeeded but response lost → next metadata check sees our own content hash and marks synced (no false conflict).

Implementation: `ConflictResolver`, `SyncEngine`, wiring into `AppViewModel`, conflict banner.

Phase 7 — Hardening & release

- T7.01 5 MB note: typing latency acceptable; save under 100 ms (performance test).
- T7.02 Full offline day simulation then reconnect (fake clock) → single correct push.
- T7.03 Two-device simulation (two engines, one fake Drive) across interleavings → no text lost in any interleaving.
- T7.04 Accessibility audit (XCUITest `performAccessibilityAudit`).
- Manual smoke checklist on real devices; README usage + install guide; tag `v1.0.0`.

What I need from you

1. Approval of this plan (or changes).
2. Confirm assumptions A1 (one note) and A2 (native iOS/iPad/Mac).
3. GitHub repo name & visibility. The GitHub CLI token in my workspace is invalid, so I'll create the repo through your signed-in browser on github.com, then push.
4. Before Phase 4: the Google Cloud OAuth client ID (steps provided).
5. To run the app on your devices: a Mac with Xcode. CI runs all automated tests on GitHub's macOS runners.